Chapter 4: Blackbox testing

Reference

This chapter refers from the book:

Mastering Software Quality Assurance: Best Practices, Tools and Techniques for Software Developers

Introduction to Software Testing

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Software testing

- Software testing is a way to assess the quality of the software and to reduce the risk of software failure in operation.
- software testing is a process which includes many different activities.
  - The test process also includes activities such as test planning, analyzing, designing, and implementing tests, reporting test progress and results, and evaluating the quality of a test object.
- Dynamic test & static test
  - dynamic testing : testing does involve the execution of the component or system being tested
  - static testing: testing does not involve the execution of the component or system being tested; e.g., reviewing work products such as requirements, user stories, and source code.
- Verification & validation:
  - Verification: testing does involve checking whether the system meets specified requirements,
  - Validation: checking whether the system will meet user and other stakeholder needs in its operational environment(s)

Software testing objectives

- To prevent defects by evaluate work products such as requirements, user stories, design, and code
- To verify whether all specified requirements have been fulfilled
- To check whether the test object is complete and validate if it works as the users and other stakeholders expect
- To build confidence in the level of quality of the test object
- To find defects and failures thus reduce the level of risk of inadequate software quality
- To provide sufficient information to stakeholders to allow them to make informed decisions, especially regarding the level of quality of the test object
- To comply with contractual, legal, or regulatory requirements or standards, and/or to verify the test object's compliance with such requirements or standards

Errors, Defects, and Failures

- A person can make an **error (mistake)**, which can lead to the introduction of a **defect (fault or bug)** in the software code or in some other related work product.
  - An error that leads to the introduction of a defect in one work product can trigger an error that leads to the introduction of a defect in a related work product.
- If a **defect** in the code is executed, this may cause a **failure**, but not necessarily in all circumstances.
  - **failures** can also be caused by environmental conditions. For example, radiation, electromagnetic fields, and pollution can cause defects in firmware or influence the execution of software by changing hardware conditions.
  - Not all unexpected test results are failures. **False positives** may occur due to errors in the way tests were executed, or due to defects in the test data, the test environment, or other testware, or for other reasons. The inverse situation can also occur, where similar errors or defects lead to **false negatives**. False negatives are tests that do not detect defects that they should have detected; false positives are reported as defects, but aren't actually defects.

Errors, Defects, and Failures (cont.)

- Errors may occur for many reasons,
  - Time pressure
  - Human fallibility
  - Inexperienced or insufficiently skilled project participants
  - Miscommunication between project participants, including miscommunication about requirements and design
  - Complexity of the code, design, architecture, the underlying problem to be solved, and/or the technologies used
  - Misunderstandings about intra-system and inter-system interfaces, especially when such intra- system and inter-system interactions are large in number
  - New, unfamiliar technologies

Seven Testing Principles

1. Testing shows the presence of defects, not their absence
   - Testing reduces the probability of undiscovered defects remaining in the software but, even if no defects are found, testing is not a proof of correctness.
2. Exhaustive testing is impossible
   - Rather than attempting to test exhaustively, risk analysis, test techniques, and priorities should be used to focus test efforts.
3. Early testing saves time and money
   - Early testing is sometimes referred to as shift left
4. Defects cluster together
   - A small number of modules usually contains most of the defects discovered during pre-release testing, or is responsible for most of the operational failures.
   - Predicted defect clusters, and the actual observed defect clusters in test or operation, are an important input into a risk analysis used to focus the test effort

Seven Testing Principles (cont.)

5. Beware of the pesticide paradox
   - If the same tests are repeated over and over again, eventually these tests no longer find any new defects. To detect new defects, existing tests and test data may need changing, and new tests may need to be written.
   - In some cases, such as automated regression testing, the pesticide paradox has a beneficial outcome, which is the relatively low number of regression defects.
6. Testing is context dependent
   - Testing is done differently in different contexts. For example, safety-critical industrial control software is tested differently from an e-commerce mobile app. As another example, testing in an Agile project is done differently than testing in a sequential software development lifecycle project
7. Absence-of-errors is a fallacy
   - it is a fallacy (i.e., a mistaken belief) to expect that just finding and fixing a large number of defects will ensure the success of a system. For example, thoroughly testing all specified requirements and fixing all defects found could still produce a system that is difficult to use, that does not fulfill the users' needs and expectations, or that is inferior compared to other competing systems.

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Test Process

- There is no one universal software test process, but there are common sets of test activities without which testing will be less likely to achieve its established objectives.
- These sets of test activities are a test process.
- Which test activities are involved in this test process, how these activities are implemented, and when these activities occur may be discussed in an organization's test strategy

Test Process in Context

Contextual factors that influence the test process for an organization, include, but are not limited to:

- Software development lifecycle model and project methodologies being used
- Test levels and test types being considered
- Product and project risks
- International Software Testing
- Business domain
- Operational constraints, including but not limited to:
  - Budgets and resources
  - Timescales
  - Complexity
  - Contractual and regulatory requirements
- Organizational policies and practices
- Required internal and external standards

Test Activities and Tasks

A test process consists of the following main groups of activities:

- Test planning
- Test monitoring and control
- Test analysis
- Test design
- Test implementation
- Test execution
- Test completion

Test Activities and Tasks (cont.)
Test planning

- Test planning involves activities that define the objectives of testing and the approach for meeting test objectives within constraints imposed by the context (e.g., specifying suitable test techniques and tasks, and formulating a test schedule for meeting a deadline).
- Test plans may be revisited based on feedback from monitoring and control activities.

Test Activities and Tasks (cont.)
Test monitoring and control

- **Test monitoring** involves the on-going comparison of actual progress against planned progress using any test monitoring metrics defined in the test plan
- **Test control** involves taking actions necessary to meet the objectives of the test plan (which may be updated over time)
- Test monitoring and control are supported by the evaluation of exit criteria, which are referred to as the definition of done in some software development lifecycle models
- **Test progress** against the plan is communicated to stakeholders in test progress reports, including deviations from the plan and information to support any decision to stop testing.

Test Activities and Tasks (cont.)
Test analysis

- During test analysis, the test basis is analyzed to identify testable features and define associated test conditions (determines "what to test")

Test analysis includes the following major activities:

- Analyzing the test basis appropriate to the test level being considered, for example:
  - Requirement specifications, such as business requirements, functional requirements, system requirements, user stories, epics, use cases, or similar work products that specify desired functional and non-functional component or system behavior
  - Design and implementation information, such as system or software architecture diagrams or documents, design specifications, call flow graphs, modelling diagrams (e.g., UML or entity-relationship diagrams), interface specifications, or similar work products that specify component or system structure
  - The implementation of the component or system itself, including code, database metadata and queries, and interfaces
  - Risk analysis reports, which may consider functional, non-functional, and structural aspects of the component or system

Test Activities and Tasks (cont.)
Test analysis

- Evaluating the test basis and test items to identify defects of various types, such as:
  - Ambiguities
  - Omissions
  - Inconsistencies
  - Inaccuracies
  - Contradictions
  - Superfluous statements

Test Activities and Tasks (cont.)
Test analysis

- Evaluating the test basis and test items to identify defects of various types, such as:
  - Ambiguities
  - Omissions
  - Inconsistencies
  - Inaccuracies
  - Contradictions
  - Superfluous statements
- Identifying features and sets of features to be tested
- Defining and prioritizing test conditions for each feature based on analysis of the test basis, and considering functional, non-functional, and structural characteristics, other business and technical factors, and levels of risks
- Capturing bi-directional traceability between each element of the test basis and the associated test conditions

Test Activities and Tasks (cont.)
Test analysis

- In some cases, test analysis produces test conditions which are to be used as test objectives in test charters.
  - **Test charters** are typical work products in some types of experience-based testing. When these test objectives are traceable to the test basis, coverage achieved during such experience-based testing can be measured.

Test Activities and Tasks (cont.)
Test design

- the test conditions are elaborated into high-level test cases, sets of high-level test cases, and other testware. So, test analysis answers the question "what to test?" while test design answers the question "how to test?"
- Test design includes the following major activities:
  - Designing and prioritizing test cases and sets of test cases
  - Identifying necessary test data to support test conditions and test cases
  - Designing the test environment and identifying any required infrastructure and tools
  - Capturing bi-directional traceability between the test basis, test conditions, & test cases
- As with test analysis, test design may also result in the identification of similar types of defects in the test basis. Also, as with test analysis, the identification of defects during test design is an important potential benefit.

Test Activities and Tasks (cont.)
Test implementation

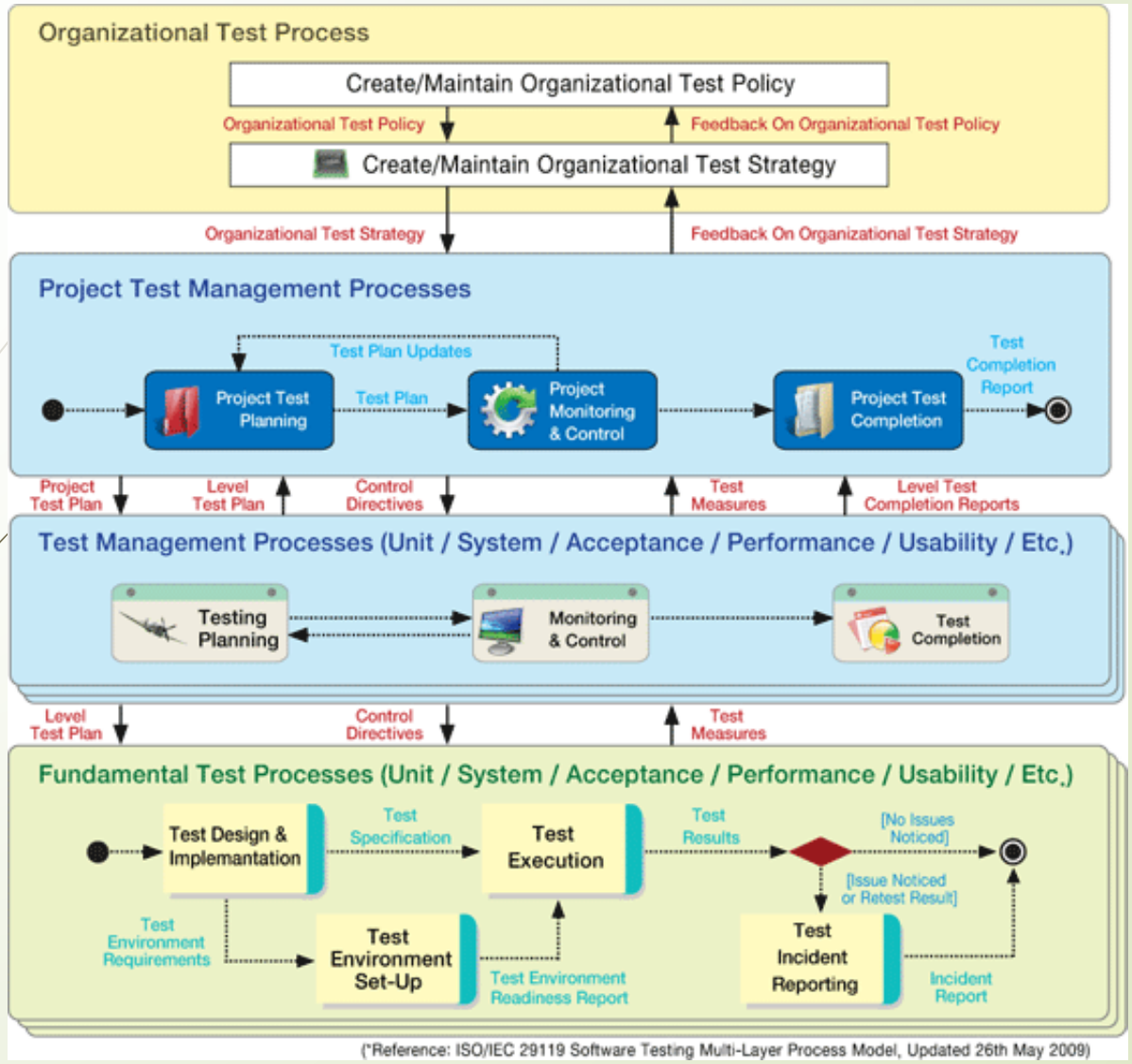
- the testware necessary for test execution is created and/or completed, including sequencing the test cases into test procedures. So, implementation answers the question "do we now have everything in place to run the tests?"
- Test implementation includes the following major activities:
  - Developing and prioritizing test procedures, and, potentially, creating automated test scripts
  - Creating test suites from the test procedures and (if any) automated test scripts
  - Arranging the test suites within a test execution schedule in a way that results in efficient test execution (see section 5.2.4)
  - Building the test environment (including, potentially, test harnesses, service virtualization, simulators, and other infrastructure items) and verifying that everything needed has been set up correctly
  - Preparing test data and ensuring it is properly loaded in the test environment
  - Verifying and updating bi-directional traceability between the test basis, test conditions, test cases, test procedures, and test suites (see section 1.4.4)
- Test design and test implementation tasks are often combined.

Test Activities and Tasks (cont.)
Test execution

- During test execution, test suites are run in accordance with the test execution schedule.
- Test execution includes the following major activities:
  - Recording the IDs and versions of the test item(s) or test object, test tool(s), and testware
  - Executing tests either manually or by using test execution tools
  - Comparing actual results with expected results
  - Analyzing anomalies to establish their likely causes (e.g., failures may occur due to defects in the code, but false positives also may occur (see section 1.2.3)
  - Reporting defects based on the failures observed (see section 5.6)
  - Logging the outcome of test execution (e.g., pass, fail, blocked)
  - Repeating test activities either as a result of action taken for an anomaly, or as part of the planned testing (e.g., execution of a corrected test, confirmation testing, and/or regression testing)
  - Verifying and updating bi-directional traceability between the test basis, test conditions, test cases, test procedures, and test results.

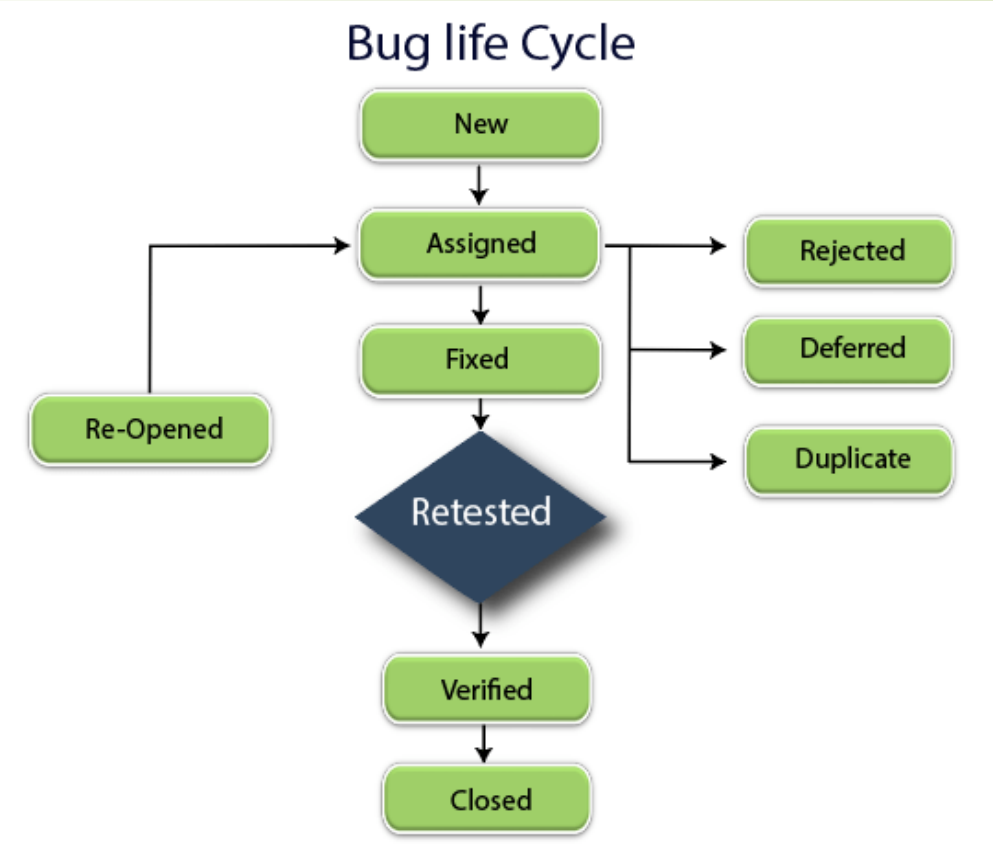
Test Activities and Tasks (cont.)
Test completion

- Test completion activities collect data from completed test activities to consolidate experience, testware, and any other relevant information. Test completion activities occur at project milestones such as when a software system is released, a test project is completed (or cancelled)
- Test completion includes the following major activities:
  - Checking whether all defect reports are closed, entering change requests or product backlog items for any defects that remain unresolved at the end of test execution
  - Creating a test summary report to be communicated to stakeholders
  - Finalizing and archiving the test environment, the test data, the test infrastructure, and other testware for later reuse
  - Handing over the testware to the maintenance teams, other project teams, and/or other stakeholders who could benefit from its use
  - Analyzing lessons learned from the completed test activities to determine changes needed for future iterations, releases, and projects
  - Using the information gathered to improve test process maturity



(*Reference: ISO/IEC 29119 Software Testing Multi-Layer Process Model, Updated 26th May 2009)

Bug management



Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Blackbox testing

- Blackbox testing is a technique for testing without knowing software source code.
- Blackbox testing (also called behavioral or behavior-based techniques) are based on an analysis of the appropriate test basis (e.g., formal requirements documents, …).
  - are applicable to both functional and non- functional testing.
  - concentrate on the inputs and outputs of the test object without reference to its internal structure.

Test Techniques

- The purpose of a test technique is to help in identifying test conditions, test cases, and test data.
- The choice of which test techniques depends on a number of factors:
  - Component or system complexity
  - Regulatory standards
  - Customer or contractual requirements
  - Risk levels and types
  - Available documentation
  - Tester knowledge and skills
  - Available tools
  - Time and budget
  - Software development lifecycle model
  - The types of defects expected in the component or system

Test Techniques (cont.)

- Some techniques are more applicable to certain situations and test levels; others are applicable to all test levels.
- When creating test cases, testers generally use a combination of test techniques to achieve the best results from the test effort.
- The use of test techniques in the test analysis, test design, and test implementation activities can range from very informal (little to no documentation) to very formal.
- The appropriate level of formality depends on the context of testing, including the maturity of test and development processes, time constraints, safety or regulatory requirements, the knowledge and skills of the people involved, and the software development lifecycle model being followed.

Blackbox test design technique

a. Equivalence Class Partitioning.
b. Boundary value analysis.
c. Decision Tables.
d. State Transition.
e. Pairwise testing.

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- **4.3.1. Equivalence partitioning technique**
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Black-box Test Techniques
Equivalence Partitioning

- Equivalence partitioning divides data into partitions (also known as equivalence classes)
  - all the members of a given partition are expected to be processed in the same way.
- Coverage is measured as the **number of equivalence partitions** tested by at least one value, divided by the **total number of identified equivalence partitions**, normally expressed as a **percentage**.
- Equivalence partitioning is applicable at all test levels.

Black-box Test Techniques
Equivalence Partitioning (cont.)

- There are equivalence partitions for both valid and invalid values.
  - **Valid values** are values that should be accepted by the component or system.
    - "valid equivalence partition": equivalence partition containing valid values
  - **Invalid values** are values that should be rejected by the component or system.
    - "invalid equivalence partition": equivalence partition containing invalid values
  - Partitions can be identified for any data element related to the test object.
    - Including: inputs, outputs, internal values, time-related values (e.g., before or after an event), interface parameters (e.g., integrated components being tested during integration testing).
  - Any partition may be divided into sub partitions if required.
  - Each value must belong to one and only one equivalence partition.
  - for invalid equivalence partitions, they should be tested individually, i.e., not combined with other invalid equivalence partitions, to ensure that failures are not masked.

Black-box Test Techniques

Equivalence Partitioning: examples

- The input condition is a range of values.
  - For example, "The value of x can only range from 0 to 100".
    - valid equivalence class is: 0 <=x <= 100
    - the two invalid equivalence classes are: x < 0 and x > 100.
- The input condition is some value.
  - Example: "Only one to six people can be registered".
    - valid equivalence class: "One to six applicants"
    - 2 classes of equivalence are invalid: "no one registered" and "more than six people registered".

Black-box Test Techniques
Equivalence Partitioning: examples

- The input condition is a set of values: We will define each value in that set as a valid equivalence class.
  - For example: “Registered vehicles are buses, coaches, trucks, taxis and motorbikes”.
    - 5 valid equivalence classes corresponding to 5 vehicle types
    - 1 class of equivalence is not valid: a vehicle other than those mentioned above eg “bicycle”.
- The entry condition is a special condition.
  - Example: “The first character must be an alphanumeric character”
    - 1 valid equivalence class: “first character is literal”
    - 1 invalid equivalence class: "not an alphanumeric character (can be a number or a special character)".

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- **4.3.2. Boundary value analysis technique**
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Black-box Test Techniques
Boundary Value Analysis

- Boundary value analysis (BVA) is an extension of equivalence partitioning, but can only be used when the partition is ordered, consisting of numeric or sequential data. The minimum and maximum values (or first and last values) of a partition are its boundary values (see Beizer 1990).
- Behavior at the boundaries of equivalence partitions is more likely to be incorrect than behavior within the partitions.
- Boundary value analysis can be applied at all test levels. This technique is generally used to test requirements that call for a range of numbers (including dates and times).
- Boundary coverage for a partition is measured as the number of boundary values tested, divided by the total number of identified boundary test values, expressed as a percentage.

Black-box Test Techniques
Boundary Value Analysis: example

- Suppose an input field accepts a single integer value as an input, using a keypad to limit inputs so that non-integer inputs are impossible. The valid range is from 1 to 5, inclusive. So, there are three equivalence partitions: invalid (too low); valid; invalid (too high). For the valid equivalence partition, the boundary values are 1 and 5. For the invalid (too high) partition, the boundary value is 6. For the invalid (too low) partition, there is only one boundary value, 0, because this is a partition with only one member.
- Some variations of this technique identify three boundary values per boundary: the values before, at, and just over the boundary. In the previous example, using three-point boundary values, the lower boundary test values are 0, 1, and 2, and the upper boundary test values are 4, 5, and 6 (see Jorgensen 2014).

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- **4.3.3. Decision table technique**
- 4.3.4. State transition testing technique
- 4.3.5. Pairwise testing technique

Black-box Test Techniques
Decision Table Testing

- Decision tables are a good way to record complex business rules that a system must implement.
- When creating decision tables, the tester identifies conditions (often inputs) and the resulting actions (often outputs) of the system.
  - These form the rows of the table, usually with the conditions at the top and the actions at the bottom.
  - Each column corresponds to a decision rule that defines a unique combination of conditions which results in the execution of the actions associated with that rule.
  - The values of the conditions and actions are usually shown as Boolean values (true or false) or discrete values (e.g., red, green, blue), but can also be numbers or ranges of numbers.
  - These different types of conditions and actions might be found together in the same table.

Black-box Test Techniques
Decision Table Testing (cont.)

- The common notation in decision tables is as follows:
  - For conditions:
    - Y means the condition is true (may also be shown as T or 1)
    - N means the condition is false (may also be shown as F or 0)
    - — means the value of the condition doesn't matter (may also be shown as N/A)
  - For actions:
    - X means the action should occur (may also be shown as Y or T or 1)
    - Blank means the action should not occur (may also be shown as – or N or F or 0)

Black-box Test Techniques
Decision Table Testing (cont.)

- A full decision table has enough columns (test cases) to cover every combination of conditions.
- The common minimum coverage standard for decision table testing is to have at least one test case per decision rule in the table. This typically involves covering all combinations of conditions.
  - Coverage is measured as the number of decision rules tested by at least one test case, divided by the total number of decision rules, expressed as a percentage.
- The strength of decision table testing :
  - helps to identify all the important combinations of conditions, some of which might otherwise be overlooked.
  - helps in finding any gaps in the requirements.
  - may be applied to all situations in which the behavior of the software depends on a combination of conditions, at any test level.

Black-box Test Techniques
Decision Table Testing: Example

Promotion for car owners if they meet at least 1 of 2 conditions: married / good student. Each input is a logical value, so the decision table only needs to have 4 columns, describing 4 different rules:

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Married?	Yes	Yes	No	No
Good Student?	Yes	No	Yes	No
Actions Discount ($)	60	25	50	0

Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- **4.3.4. State transition testing technique**
- 4.3.5. Pairwise testing technique

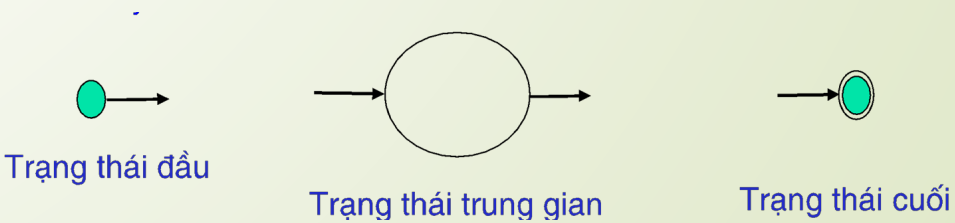
Black-box Test Techniques
State Transition Testing

- A state transition diagram shows the possible software states, as well as how the software enters, exits, and transitions between states.
  - A transition is initiated by an event (e.g., user input of a value into a field). The event results in a transition.
  - The same event can result in two or more different transitions from the same state.
  - The state change may result in the software taking an action (e.g., outputting a calculation or error message).
- A state transition table shows all valid transitions and potentially invalid transitions between states, as well as the events, and resulting actions for valid transitions.
- State transition diagrams normally show only the valid transitions and exclude the invalid transitions.

Black-box Test Techniques
State Transition Testing (cont.)

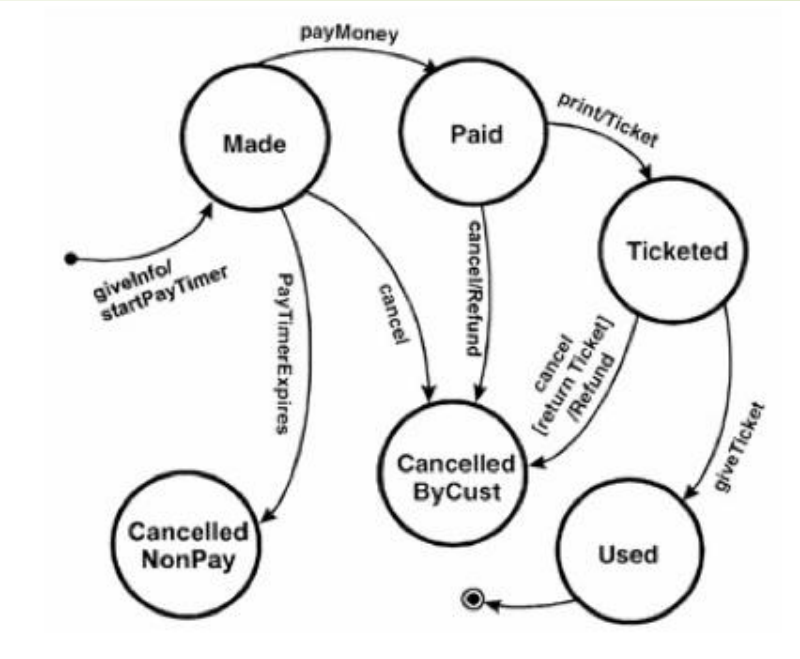
- Tests can be designed to cover a typical sequence of states, to exercise all states, to exercise every transition, to exercise specific sequences of transitions, or to test invalid transitions.
- Application:
  - for menu-based applications and is widely used within the embedded software industry.
  - modeling a business scenario having specific states or for testing screen navigation. The concept of a state is abstract – it may represent a few lines of code or an entire business process.
- Coverage is commonly measured as the number of identified states or transitions tested, divided by the total number of identified states or transitions in the test object, expressed as a percentage.

Black-box Test Techniques
State Transition Testing (cont.)



Black-box Test Techniques
State Transition Testing: Example

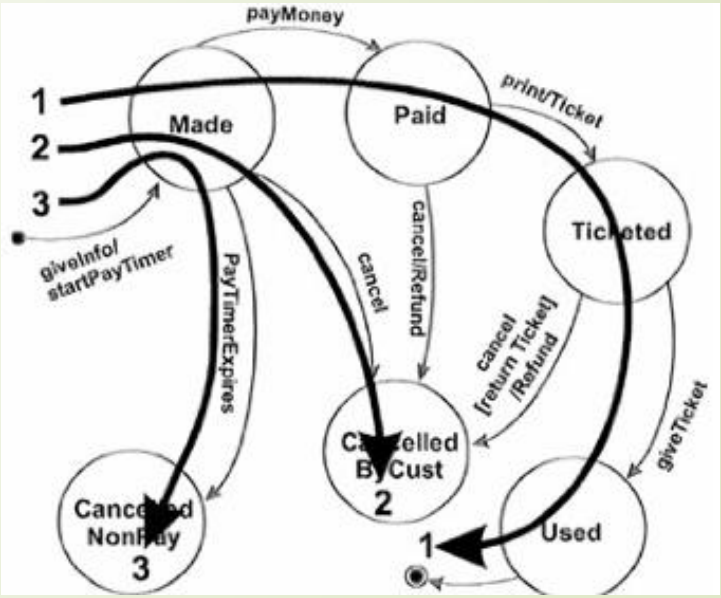
Air ticket booking module has 6 status



Black-box Test Techniques

State Transition Testing: Example (cont.)

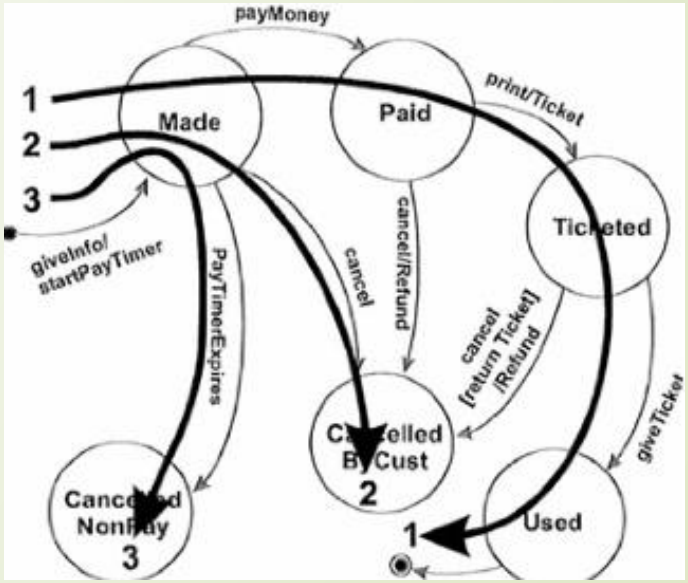
- **Coverage level 1 :** Create test cases such that each state occurs at least once. For example, 3 paths will reach level 1 . coverage



Black-box Test Techniques

State Transition Testing: Example (cont.)

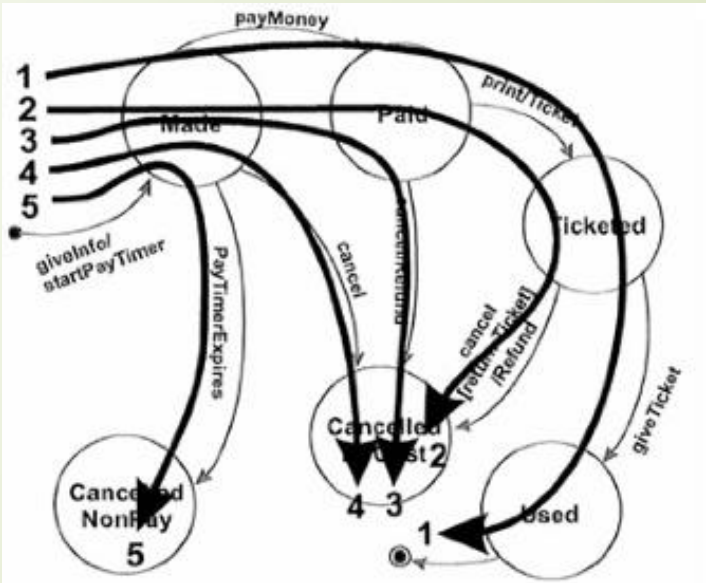
- **Coverage level** 2 :
  Create test cases such that each event occurs at least once. For example, 3 paths reach level 2 . coverage



Black-box Test Techniques

State Transition Testing: Example (cont.)

- Testing level 3 : create paths such that all transition paths are tested. A transition path is a defined state transition, starting from the input state and ending at the end state.
  - This is the best coverage result because it exhausts all possibilities, but it's not feasible when the transition path has loops.
- Testing level 4 : create paths such that all linear transition paths are tested.



Outline

4.1. Definition and objectives

4.2. Software testing process

4.3. Blackbox testing techniques

- 4.3.1. Equivalence partitioning technique
- 4.3.2. Boundary value analysis technique
- 4.3.3. Decision table technique
- 4.3.4. State transition testing technique
- **4.3.5. Pairwise testing technique**

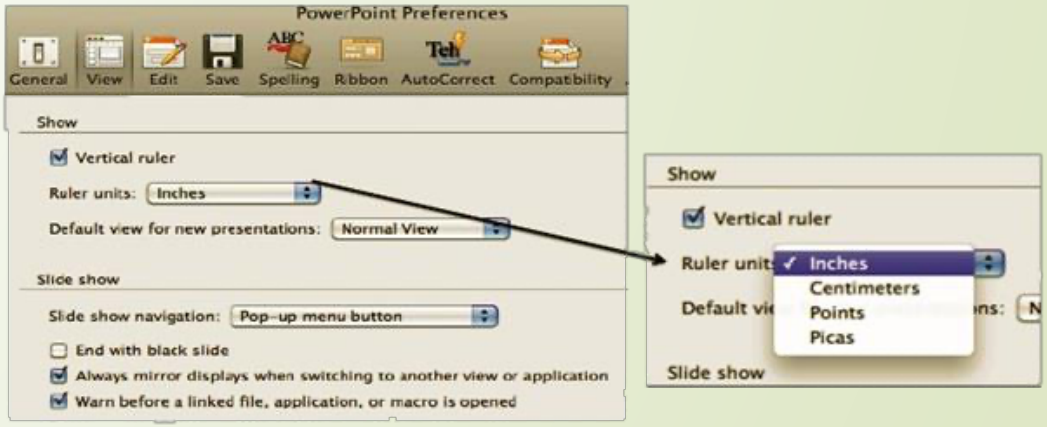
Black-box Test Techniques

Pairwise testing

- In fact, most errors are generated from the combination of values of input parameter pairs.
- Method:
  - Selection of input parameters and corresponding values
  - Get the combination (pairwise) of values between 2 parameters
  - Build the test set so that it covers all the pairs defined above

Black-box Test Techniques
Pairwise testing: Example

- Consider the View options tab from a version of Microsoft Powerpoint software



(a) (b)

Vertical Ruler	Ruler Units	Default View	SS Navigation	End with Black	Always Mirror	Warn Before
Visible	Inches	Normal	Pop-up	Yes	Yes	Yes
Invisible	Centimeters	Slide	None	No	No	No
	Points	Outline				
	Picas					

(c)

Black-box Test Techniques

Pairwise testing: Example (cont.)

- Tab View_preference has 7 atributes, each atributes has several values: Vertical_Ruler (Visible, InVisible), Ruler_units (Inches, Centimetes, Points, Picas), Default_View (Normal, Slide, Outline), Ss_Navigator (Popup, None), End_With_Black (Yes1, No1), Always_Mirror (Yes2, No2), Warn_Before (Yes3, No3).
- If test all cases, we have: 2*4*3*2*2*2*2 = 384 test case.
- The possible pairs: (Visible, *Inches*), (Visible, *Centimetes*),… , (No2, *No3*)
- Test case *(Visible, Centimetes, Nomal, Non, No1, Yes2, Yes3) includes* 21 pairs *(Visible, Centimetes), (Visble, Nomal),…, (Yes2, Yes3)*

Black-box Test Techniques

Pairwise testing: Example (cont.)

- A test suite coveres all possible pairs (created by tool pairwise PICT http://download.microsoft.com/download/f/5/5/f55484df-8494-48fa-8dbd-8c6f76cc014b/pict33.msi)

Vertical Ruler	Ruler units	Default View	Ss Navigator	End Black	With Always Mirror	Warn Before
Visible	Centimetes	Normal	None	No1	Yes2	Yes3
InVisible	Picas	Normal	Popup	Yes1	No2	No3
Visible	Picas	Slide	Popup	No1	No2	Yes3
InVisible	Inches	Normal	None	Yes1	Yes2	No3
InVisible	Points	Outline	None	No1	No2	No3
Visible	Centimetes	Slide	Popup	Yes1	Yes2	No3
Visible	Picas	Outline	None	Yes1	Yes2	Yes3
InVisible	Inches	Outline	Popup	No1	No2	Yes3
Visible	Points	Slide	None	Yes1	Yes2	Yes3
InVisible	Inches	Slide	None	No1	Yes2	Yes3
Visible	Inches	Normal	Popup	No1	Yes2	Yes3
Visible	Points	Normal	Popup	Yes1	Yes2	No3
InVisible	Centimetes	Outline	Popup	Yes1	No2	Yes3